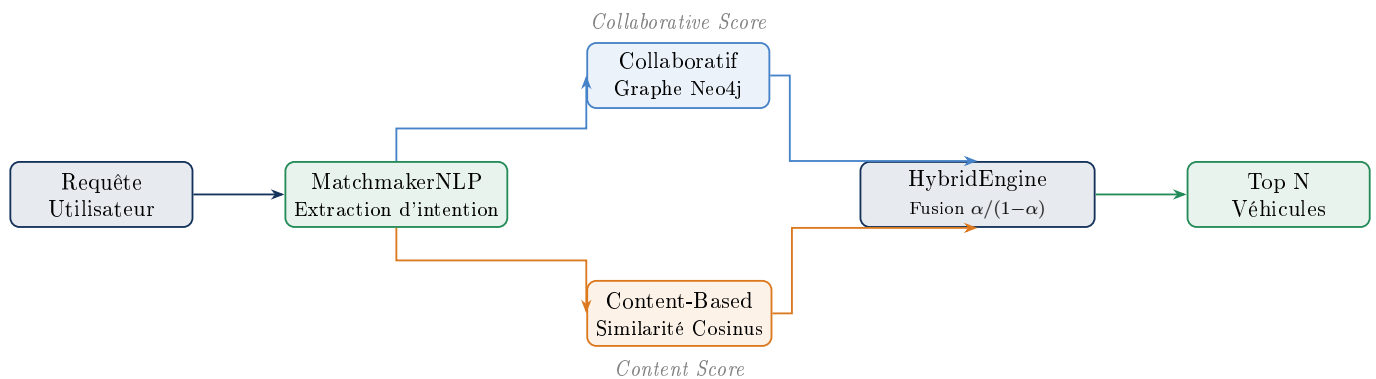


WAKALA

Plongée au Cœur du Moteur de Recommandation Hybride

Introduction

Le moteur de recommandation de la plateforme *Wakala* est conçu pour offrir aux acheteurs les véhicules les plus pertinents par rapport à leurs besoins et comportements. Contrairement à une simple recherche par filtres, notre système s'appuie sur une approche **hybride** qui combine la similarité des caractéristiques des véhicules (Content-Based) et les comportements des utilisateurs (Filtrage Collaboratif via Graphe).



Vue d'ensemble : les quatre étapes du moteur hybride Wakala.

Composants du Moteur de Recommandation

L'architecture du moteur repose sur quatre piliers fondamentaux :

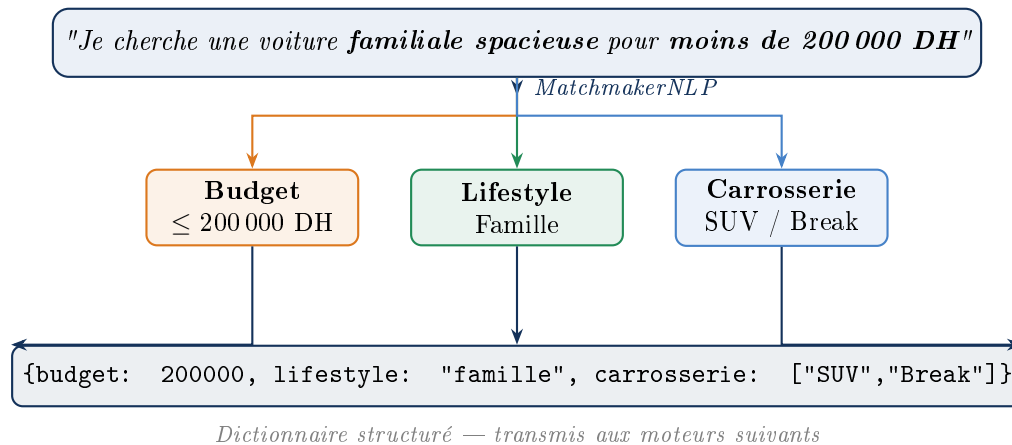
- **MatchmakerNLP** : Extraction sémantique des intentions utilisateur.
- **Filtrage Basé sur le Contenu (Content-Based)** : Similarité Cosinus mathématique.
- **Filtrage Collaboratif (Neo4j)** : Analyse comportementale en graphe.
- **Moteur Hybride (HybridEngine)** : Agrégation intelligente avec A/B testing.

1. Matchmaker NLP : Comprendre l'Utilisateur

Avant de chercher un véhicule, le système doit comprendre des requêtes en langage naturel telles que *"Je cherche une voiture familiale spacieuse pour moins de 200 000 DH"*. Le composant **MatchmakerNLP** convertit ce texte libre en un dictionnaire de caractéristiques structurées.

- **Extraction Budgétaire** : Utilisation d'expressions régulières (Regex) pour capturer les montants numériques associés aux mots "budget", "moins de" ou "max".
- **Classification de Style de vie (Lifestyle)** : Détection de mots-clés :
 - *Famille* ("enfants", "minivan", "spacieux") → Filtres : SUV, Break.
 - *Urbain* ("ville", "citadine") → Filtres : Citadine, Compacte.

- *Écologique* ("hybride", "électrique").
- *Sportif* ("rapide", "puissant").



2. Filtrage Basé sur le Contenu (Content-Based)

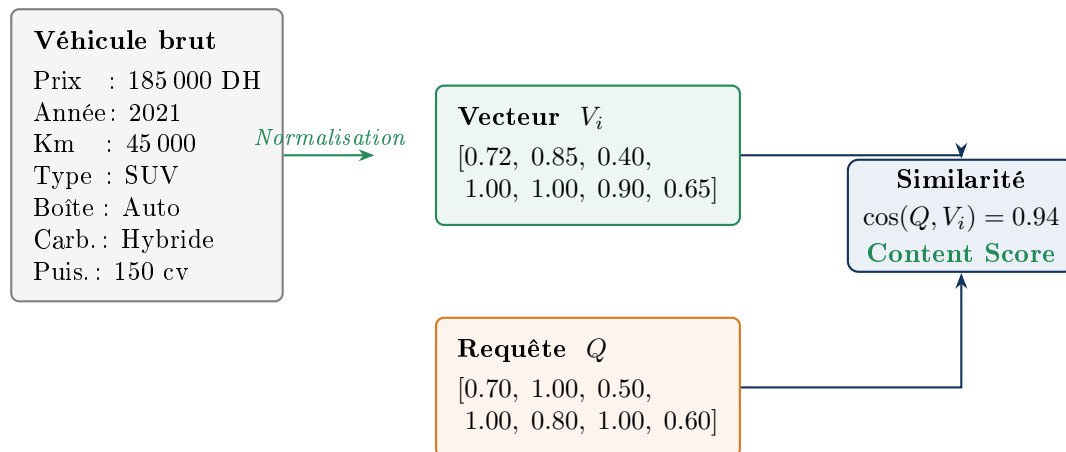
Le composant `content_based.py` évalue mathématiquement à quel point un véhicule correspond aux critères de l'utilisateur.

Vectorisation et Normalisation

Chaque véhicule est transformé en un vecteur de 7 dimensions (les *Features*) :

$$V_i = [\text{Prix}, \text{Année}, \text{Kilométrage}, \text{Type}, \text{Boîte}, \text{Carburant}, \text{Puissance}] \quad (1)$$

Les données ayant des échelles très différentes, le système applique des algorithmes de normalisation (`StandardScaler` et `MinMaxScaler` de *scikit-learn*) pour ramener toutes les valeurs sur des échelles comparables.



Un score proche de 1 signifie que le véhicule correspond presque parfaitement à la requête.

Similarité Cosinus

Le système construit un vecteur requête Q basé sur les critères de l'utilisateur, et calcule la **Similarité Cosinus** entre Q et chaque véhicule V_i :

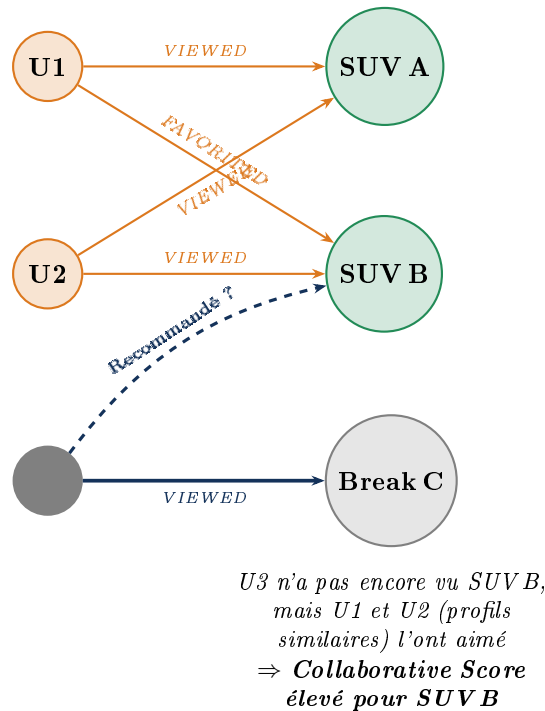
$$\text{Similarité}(Q, V_i) = \frac{Q \cdot V_i}{\|Q\| \times \|V_i\|} \quad (2)$$

Le résultat est normalisé entre 0 et 1 pour générer le **Content Score**.

3. Filtrage Collaboratif (Neo4j)

L'approche basée sur le contenu ne suffit pas toujours. Si un utilisateur cherche un SUV, il est pertinent de lui recommander les SUV que les autres utilisateurs au profil similaire ont consultés ou mis en favoris.

La base de données **Neo4j** stocke les entités (Users, Vehicles) sous forme de Nœuds, et les interactions sous forme de Relations (**VIEWED**, **FAVORITED**). En interrogeant le graphe, le système génère un **Collaborative Score** mesurant la popularité comportementale d'un véhicule par rapport à la "bulle" de l'utilisateur.



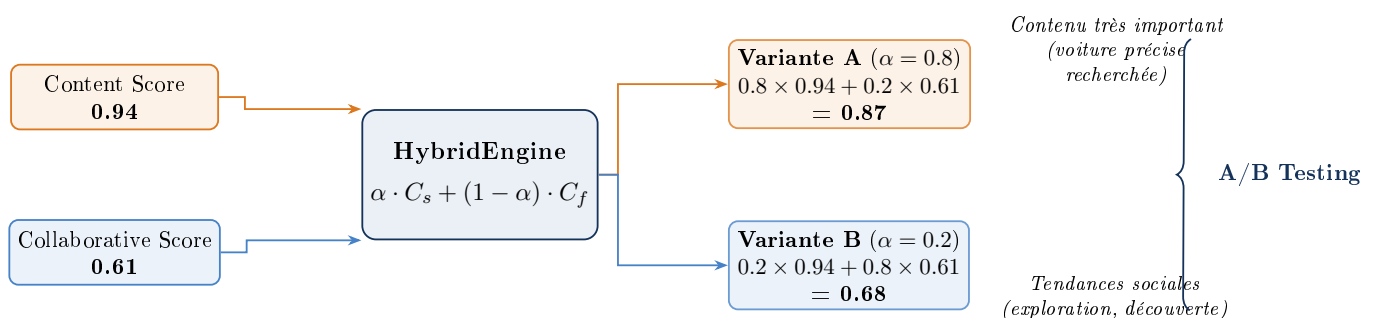
4. Le Moteur Hybride et l'A/B Testing

Le **HybridEngine** fusionne les résultats des deux méthodes précédentes via une pondération :

$$\text{Score Final} = \alpha \times \text{ContentScore} + (1 - \alpha) \times \text{CollaborativeScore} \quad (3)$$

Pour optimiser l'algorithme, le système effectue de l'A/B Testing en assignant aléatoirement les utilisateurs connectés dans deux seaux :

- **Variante A** ($\alpha = 0.8$) : Privilégie massivement le moteur *Content-Based* (caractéristiques techniques du véhicule).
- **Variante B** ($\alpha = 0.2$) : Privilégie massivement le moteur *Collaboratif* (comportement social et tendances).



Le système mesure lequel des deux α génère le plus de clics et de conversions, puis s'adapte automatiquement.

Synthèse

Le Moteur de Recommandation de la plateforme *Wakala* orchestre quatre composants complémentaires pour transformer une requête en langage naturel en une liste de véhicules classés par pertinence. Le tableau ci-dessous récapitule le rôle, les données mobilisées et la sortie de chaque étage du pipeline.

Composant	Rôle	Données mobilisées	Sortie
MatchmakerNLP	Extraction sémantique de l'intention utilisateur (budget, style de vie, carrosserie)	Requête en langage naturel	Dictionnaire structuré
Content-Based	Similarité mathématique entre le véhicule et la requête	Vecteur V_i à 7 dimensions, normalisation <i>StandardScaler</i> / <i>MinMaxScaler</i>	Content Score $\in [0, 1]$
Filtrage Collaboratif (Neo4j)	Analyse comportementale via le graphe des interactions	Nœuds Users/Vehicles, relations VIEWED / FAVORITED	Collaborative Score $\in [0, 1]$
HybridEngine	Fusion pondérée des deux scores et arbitrage A/B	Content Score, Collaborative Score, coefficient α	Top N véhicules classés

Ce qu'apporte l'approche hybride. En combinant le NLP, la Similarité Cosinus vectorielle et la puissance de la théorie des Graphes, le système évite les deux écueils classiques des moteurs de recommandation mono-approche : la surspécialisation du *Content-Based* (qui ne recommande que des véhicules « techniquement » proches et peine à sortir des sentiers battus) et la dépendance du *Collaboratif* aux historiques riches (peu efficace pour un nouvel utilisateur ou un véhicule tout juste mis en ligne). L'infrastructure d'A/B Testing native permet en outre d'ajuster empiriquement le curseur α plutôt que de figer une pondération arbitraire, et de faire évoluer cet équilibre à mesure que le comportement des utilisateurs de la plateforme change.

Concrètement, les deux moteurs ne dépendent pas des mêmes données et se relaient donc naturellement selon le profil de l'utilisateur. Un acheteur qui soumet sa toute première requête obtient malgré tout un Content Score fiable, car celui-ci ne repose que sur les caractéristiques intrinsèques du véhicule (prix, kilométrage, carrosserie...) comparées à sa requête — aucun historique n'est nécessaire. Son Collaborative Score, en revanche, sera nul ou proche de zéro faute d'interactions passées à exploiter dans le graphe. À l'inverse, un utilisateur récurrent, dont le profil recoupe celui d'autres acheteurs actifs, bénéficie d'un Collaborative Score affiné qui capture des signaux invisibles au seul filtrage par attributs : un véhicule plébiscité par des profils similaires peut ainsi remonter dans le classement même s'il n'est pas la correspondance technique la plus exacte. Le HybridEngine ne tranche pas entre ces deux logiques, il les additionne selon α — ce qui explique pourquoi la Variante A ($\alpha = 0.8$, voir Section 4) reste pertinente pour les nouveaux utilisateurs, tandis que la Variante B ($\alpha = 0.2$) valorise davantage les tendances collectives pour les profils établis.

Limites actuelles et pistes d'évolution. Quatre axes de progression se dégagent naturellement de l'architecture actuelle :

- **Cold start.** Le Collaborative Score repose sur la traversée du graphe Neo4j via les relations VIEWED / FAVORITED déjà enregistrées : sans arête existante, aucun score ne peut être calculé, qu'il s'agisse d'un utilisateur qui vient de s'inscrire ou d'un véhicule tout juste publié. Une stratégie de repli (moyenne du segment démographique le plus proche, popularité globale du véhicule, ou pondération α

automatiquement portée à 1 en l'absence de signal collaboratif) permettrait de lisser cette transition plutôt que de renvoyer un score arbitrairement nul.

- **Pondération dynamique.** Le partage actuel en deux seaux fixes ($\alpha = 0.8$ / $\alpha = 0.2$) impose de choisir une variante *a priori* et d'attendre la fin d'une campagne pour en tirer une conclusion globale. Un algorithme de bandit contextuel (*multi-armed bandit*) remplacerait ce test binaire par un ajustement continu : α serait recalculé à chaque requête en fonction du contexte (nouveau vs. utilisateur récurrent, catégorie de véhicule recherchée), et la part de trafic allouée à chaque valeur d' α évoluerait automatiquement vers celles qui génèrent le plus de clics, sans attendre la clôture d'une campagne A/B.
- **Enrichissement sémantique.** Le MatchmakerNLP actuel repose sur des règles déterministes (Regex, listes de mots-clés par catégorie de style de vie) : il ne reconnaît que les formulations anticipées à l'avance et ignore les synonymes ou tournures imprévues (« voiture pour la famille » vs. « véhicule pour les enfants »). Une couche d'embeddings sémantiques (par exemple des *sentence embeddings*) comparerait le sens des phrases plutôt que leur forme littérale, élargissant la couverture des requêtes libres sans qu'il soit nécessaire d'énumérer manuellement chaque variante lexicale.
- **Passage à l'échelle et explicabilité.** À mesure que le graphe Neo4j grossit (plus d'utilisateurs, plus de véhicules, plus d'interactions), le calcul du Collaborative Score en temps réel devient plus coûteux ; un pré-calcul périodique (batch) des scores les plus consultés, mis en cache, limiterait la latence perçue. Par ailleurs, plus les scores combinent de sources hétérogènes, plus il devient utile d'exposer à l'utilisateur *pourquoi* un véhicule est recommandé (ex. « proche de votre budget » vs. « apprécié par des profils similaires »), afin de garder le système compréhensible et digne de confiance.

En l'état, l'architecture reste néanmoins un socle solide et évolutif : chaque composant (NLP, Content-Based, Collaboratif, Hybride) est découplé des autres et peut être amélioré, remplacé ou réentraîné indépendamment, sans remettre en cause l'ensemble du pipeline.